

Contagem de Primos e Race Conditions

Paralelismo de Laços e Cláusula Reduction com OpenMP

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

August 22, 2026

1 Enunciado

Implementar um programa em C que conte quantos números primos existem entre 2 e um valor máximo N . Depois, paralelizar o laço principal usando a diretiva `#pragma omp parallel for` sem alterar a lógica original. Comparar o tempo de execução e os resultados das versões sequencial e paralela. Observar possíveis diferenças no resultado e no desempenho, refletindo sobre desafios como correção e distribuição de carga.

2 Fundamentação teórica

2.1 Paralelismo de Laços (Loop Parallelism)

Na programação paralela, é comum distribuir iterações independentes de um laço `for` entre diferentes threads. A diretiva `#pragma omp parallel for` divide as iterações automaticamente. Sob o escalonamento padrão (`static`), o compilador divide os índices em blocos iguais e os distribui para as threads em tempo de compilação.

2.2 Condição de Corrida (Race Condition)

Uma *Race Condition* surge quando múltiplas threads operam sem sincronização sobre uma mesma variável em memória compartilhada, e pelo menos uma altera o dado. A CPU não incrementa `count++` em um único ciclo. O hardware quebra a instrução em três passos: lê a memória para o registrador (`LOAD`), soma no registrador (`ADD`) e escreve de volta na memória (`STORE`). Se duas threads executam o `LOAD` juntas, ambas leem o mesmo número. Elas somam 1 localmente e escrevem o mesmo resultado. O programa perde um incremento e retorna um valor incorreto.

2.3 Reduction

Se o código travar a variável global com `critical` em cada iteração, as threads formarão uma fila e a execução ficará mais lenta que a versão sequencial. A diretiva `reduction(op:var)` resolve isso criando uma cópia privada da variável para cada thread no escopo do laço, com valor zero. As threads incrementam suas cópias locais na cache sem travar o barramento. Ao final do bloco paralelo, o OpenMP soma os subtotais na variável global.

3 Experimento

3.1 Configuração

O código utiliza $N = 5.000.000$ elementos. A função verifica os divisores ímpares até $\sqrt{n}$. Testar números maiores consome mais ciclos de CPU. Isso desequilibra a carga de trabalho no laço.

O script executa três versões com as flags `-O3` e `-fopenmp`:

1. **Sequencial:** 1 thread. Sem OpenMP.
2. **OpenMP Naive:** Usa `#pragma omp parallel for`. As threads modificam a variável global de contagem `count` causando corrupção de memória.
3. **OpenMP Correct:** Usa `reduction(+:count)` protegendo o somatório.

4 Código-fonte

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6 #include <omp.h>
7
8 #define DEFAULT_N 5000000L
9
10 static double agora(void) {
11     struct timespec t;
12     clock_gettime(CLOCK_MONOTONIC, &t);
13     return t.tv_sec + t.tv_nsec / 1e9;
14 }
15
16 int is_prime(long n) {
17     if (n <= 1) return 0;
18     if (n == 2) return 1;
19     if (n % 2 == 0) return 0;
20     for (long i = 3; i * i <= n; i += 2) {
21         if (n % i == 0) return 0;
22     }
23     return 1;
24 }
25
26 int main(int argc, char **argv) {
27     long n = argc > 1 ? atol(argv[1]) : DEFAULT_N;
28     if (n <= 0) return 1;
29
30     int n_threads = omp_get_max_threads();
31     double inicio_seq = agora();
32     long count_seq = 0;
33 }
```

```

34     for (long i = 2; i <= n; ++i) {
35         if (is_prime(i)) {
36             count_seq++;
37         }
38     }
39     double tempo_seq = agora() - inicio_seq;
40
41     double inicio_omp_naive = agora();
42     long count_omp_naive = 0;
43
44     #pragma omp parallel for
45     for (long i = 2; i <= n; ++i) {
46         if (is_prime(i)) {
47             count_omp_naive++;
48         }
49     }
50     double tempo_omp_naive = agora() - inicio_omp_naive;
51
52     double inicio_omp_red = agora();
53     long count_omp_red = 0;
54
55     #pragma omp parallel for reduction(+:count_omp_red)
56     for (long i = 2; i <= n; ++i) {
57         if (is_prime(i)) {
58             count_omp_red++;
59         }
60     }
61     double tempo_omp_red = agora() - inicio_omp_red;
62
63     printf("N=%ld threads=%d seq_time=%.9f omp_naive_time=%.9f\n",
64           omp_red_time=%.9f seq_count=%ld omp_naive_count=%ld omp_red_count=%ld\n",
65           n, n_threads, tempo_seq, tempo_omp_naive, tempo_omp_red,
66           count_seq, count_omp_naive, count_omp_red);
67
68     return 0;
69 }

```

Listing 1: tarefa05.c

5 Resultados

A execução no shell (variando de 1 a 28 threads) obteve o seguinte perfil para a busca de primos até $N = 5.000.000$.

5.1 Speedup e Observação de Race Conditions

O programa identificou 348.513 primos nas versões Sequencial e Reduction. A versão Naive falhou: rodando em 2 threads relatou apenas 165.441, e em 8 threads caiu para 40.786. Os incrementos se sobreescreveram milhares de vezes na memória.

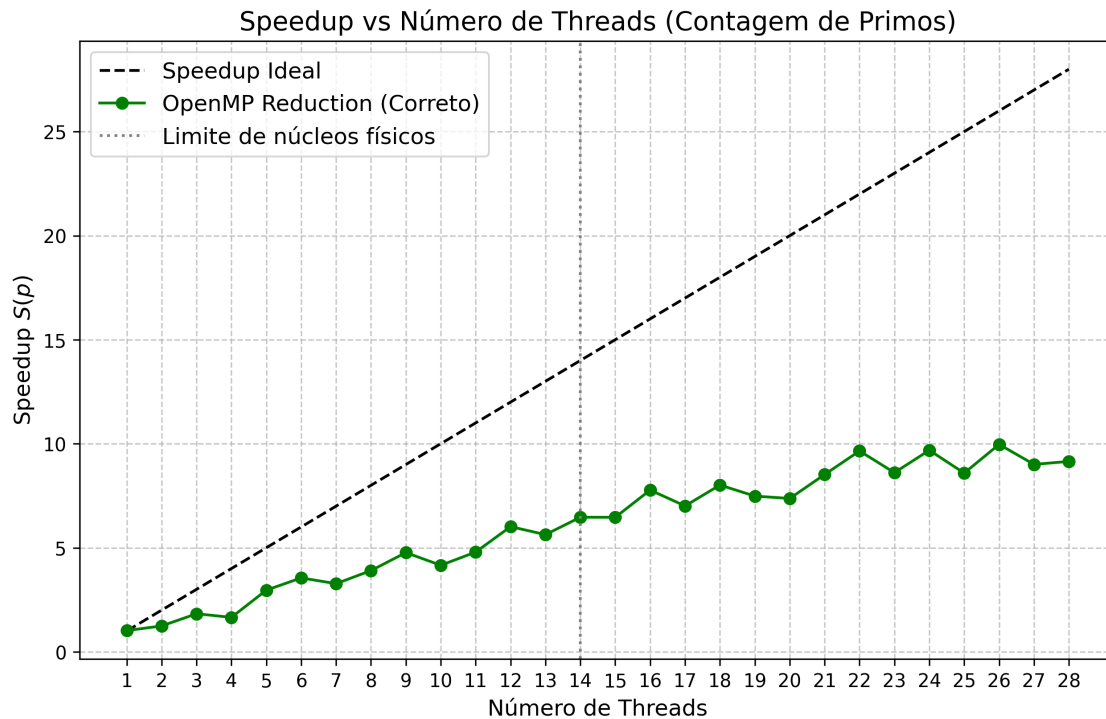


Figure 1: Gráfico de Speedup para a contagem de primos com Reduction. O desbalanceamento de carga (testar números maiores demora mais) impede que a curva atinja a diagonal ideal linear (linha preta tracejada).

6 Discussão

Este problema roda estritamente dentro da CPU e da cache L1. É totalmente CPU-bound e deve escalar. Contudo, há dois problemas mecânicos:

6.1 Erros Silenciosos

Falhas de memória geralmente quebram a aplicação (*segmentation fault*). As Race Conditions não. O software compila, executa rápido e imprime um número no terminal. Só sabemos que o resultado está errado porque comparamos com o sequencial. É o programador quem deve garantir a corretude usando reduções locais ou estados imutáveis, em vez de compartilhar contadores nuzos.

6.2 O Desbalanceamento de Carga (Load Imbalance)

A Figura 1 mostra que a linha verde fica abaixo da reta tracejada de speedup perfeito. A responsabilidade disso recai sobre o escalonamento estático do compilador.

Se o OpenMP quebra um vetor de 10.000 iterações no meio, a Thread 0 julga os menores números e resolve tudo muito rápido. A Thread 1 fica com números maiores, roda divisões longas, e passa muito mais tempo na ALU. A Thread 0 termina o lote e fica ociosa esperando, desperdiçando potencial do processador.

7 Conclusão

O `#pragma omp parallel for` expõe as threads sem que precisemos reescrever a lógica interna do `for`. Ainda assim, precisamos tratar a memória usando `reduction` e balancear a carga. Em problemas onde as iterações têm durações diferentes, o escalonador `static` perde eficiência. Para corrigir o problema na Figura 1, o código pode usar escalonamento dinâmico (`schedule(dynamic, 100)`). Assim, as threads que acabam mais rápido buscam mais lotes de trabalho, evitando ociosidade.